

I/O ve Disk Zamanlaması

CSE312 Operating Systems — Lecture 08

Kapsam	Sayfa 1-18 — Elevator Algorithm dahil
Hazırlayan	CSE312 Ders Notları — Spring 2026
Konular	I/O Modelleri, Cihaz Sürücüleri, Disk Zamanlaması

İçindekiler

- 1 I/O ve İşletim Sistemleri — Genel Bakış
- 2 I/O Cihaz Türleri
- 3 I/O Modelleri (Unix, Windows, Network)
- 4 Unix Cihaz Dosyaları — Major/Minor Numaraları
- 5 Device-Specific Semantics ve ioctl
- 6 Cihaz Sürücü Arayüzü (Device Driver Interface)
- 7 Blok Cihazlar (Block Devices)
- 8 Disk Zamanlaması — Giriş
- 9 Çatışan Gereksinimler (Conflicting Demands)
- 10 FCFS — İlk Gelen İlk Hizmet
- 11 SSF — En Kısa Seek Önce
- 12 Elevator Algorithm (SCAN)
- 13 Quiz Soruları ve Cevapları
- 14 Kitap Bölüm Sonu Soruları ve Çözümleri

1. I/O ve İşletim Sistemleri — Genel Bakış

Her türlü giriş/çıkış (I/O) işlemi işletim sistemi üzerinden geçmek zorundadır. Modern bir bilgisayar sistemi disk sürücüler, yazıcılar, ağ kartları, ekran kartları, klavye/fare gibi onlarca farklı cihaz türünü barındırabilir. İşletim sistemi tüm bu çeşitliliği uygulamadan gizleyerek tutarlı bir arayüz sunar.

i Temel Prensiptir

Tüm I/O işlemleri işletim sistemi üzerinden akar. Bu sayede uygulama programcısı donanım detaylarıyla uğraşmak zorunda kalmaz.

Temel Sorular

- **Bu cihazlarla hangi yazılım konuşur?**
 - Cihaz sürücüler (device drivers) — çekirdek içinde çalışan, donanıma özel kod parçaları.
- **OS ile bu yazılım arasındaki arayüz nedir?**
 - Standart entry point'ler: open, close, read, write, ioctl gibi fonksiyonlar.
- **Uygulama programlarına arayüz nedir?**
 - POSIX/Windows API katmanı; uygulamalar cihazları dosya gibi kullanır.
- **Donanım kısıtlamaları var mı?**
 - Evet — DMA, interrupt, port I/O, memory-mapped I/O gibi donanım mekanizmaları sürücüler tarafından yönetilir.

□ Tasarım Hedefi

Hedef: Çoğu uygulamanın cihazdan bağımsız (device-independent) çalışmasını sağlayan bir I/O modeli oluşturmak.

2. I/O Cihaz Türleri

Gerçek sistemlerdeki cihazlar, veri hızları ve erişim şekilleri bakımından muazzam bir yelpazede yer alır. Aşağıdaki tablo tipik cihaz, ağ ve bus veri hızlarını özetlemektedir:

Cihaz / Bağlantı	Tipik Veri Hızı	Notlar
Klavye (keyboard)	~10 byte/sn	Karakter odaklı, çok yavaş
Seri Port (serial port)	~1 KB/sn	Eski terminal bağlantıları
USB 2.0 flash sürücü	~60 MB/sn	Block cihaz
SATA SSD	~500 MB/sn	Block cihaz, düşük gecikme
NVMe SSD	~3-7 GB/sn	PCIe, çok düşük gecikme
Gigabit Ethernet	~125 MB/sn	Network cihazı, paket tabanlı
10 Gigabit Ethernet	~1.25 GB/sn	Veri merkezi bağlantısı
Grafik Kartı (PCIe x16)	~16 GB/sn	Frame buffer, bitmap

⚠ Neden Farklı Modeller?

Veri hızlarındaki bu devasa fark (10 byte/sn ile 16 GB/sn arasında), işletim sisteminin her cihazı ayrı bir stratejiyle yönetmesini zorunlu kılar. Disk zamanlaması tam da bu yüzden kritik bir optimizasyon konusudur.

3. I/O Modelleri

İşletim sistemleri cihazları birkaç temel kategoriye ayırarak yönetir. Bu kategoriler hem kernel içindeki yapıyı hem de uygulama API'sini şekillendirir.

3.1 Unix Modeli

Unix, cihazları iki ana kategoriye ayırır:

Kategori	Tanım	Örnekler	Uygulama erişimi
Block Devices (Blok Cihazlar)	Sabit boyutlu bloklar halinde veri okuma/yazma	Hard disk, SSD, CD-ROM	Dosya sistemi üzerinden (nadiren doğrudan)
Character Devices (Karakter Cihazlar)	Yapılandırılmamış byte dizisi döndürür	Klavye, seri port, terminal, ses kartı	Doğrudan read/write çağrıları

Bu iki kategori **yüksek derecede cihaz bağımsızlığı (device independence)** sağlar — çoğu uygulama, arkasındaki donanımın hangisi olduğunu bilmeden çalışabilir.

3.2 Windows Modeli

Windows, ekranları ve yazıcıları bitmap olarak ele alır. Bu yaklaşım grafiksel uygulamalar için daha doğal bir model sunar:

- Ekranlar (screens) → bitmap
- Yazıcılar (printers) → bitmap (GDI / PDL üzerinden)
- Diskler → Unix ile aynı blok modeli

3.3 Network Cihazları

⚠ Özel Durum: Network

Ağ cihazları ne blok ne de karakter-odaklıdır. Paket tabanlı (packet-based) çalışırlar ve işletim sistemi tarafından tamamen farklı bir alt sistemle (network stack) yönetilirler. Socket API'si ağ cihazlarını soyutlar.

I/O Modeli	Kullanım Alanı	OS Desteği
Character Device	Terminal, seri port, ses	Unix/Linux, Windows
Block Device	Disk, flash depolama	Unix/Linux, Windows
Bitmap	Ekran, yazıcı	Önce Windows, sonra her yerde
Network	Ethernet, Wi-Fi	Tüm modern OS'ler

4. Unix Cihaz Dosyaları — Major/Minor Numaraları

Unix'in en güçlü soyutlamalarından biri, cihazlara dosya sistemi üzerinden erişebilmektir. **/dev** dizini altındaki her dosya gerçekte bir cihazı temsil eder.

```
$ ls --time-style="+" -Ggl /dev/sda1 /dev/pts/6
crw--w---- 1 136, 6 /dev/pts/6
brw-rw---- 1 8, 1 /dev/sda1
```

ls çıktısı: c=karakter cihaz, b=blok cihaz

Çıktı Açıklaması

Alan	Değer (pts/6)	Değer (sda1)	Anlam
İlk harf	c	b	c=character, b=block
Major numarası	136	8	Cihaz türü / sürücü seçimi
Minor numarası	6	1	Birim / bölüm (partition)
Dosya adı	/dev/pts/6	/dev/sda1	pseudo-terminal / 1. SCSI disk bölümü

4.1 Major Device Numbers

Major numaralar **cihaz türünü** — daha doğrusu hangi **cihaz sürücüsünün** kullanılacağını — belirtir. Çekirdek, major numarasına bakarak ilgili sürücü fonksiyonlarını çağırır. Numaralandırma sisteme özgüdür (Linux, macOS farklı).

i Ayır Namespace

Blok ve karakter cihazlar birbirinden bağımsız major numarası namespace'ine sahiptir. Yani hem bir blok cihaz hem bir karakter cihaz aynı major numarasına sahip olabilir — çakışma yoktur.

4.2 Minor Device Numbers

Minor numaralar **cihaz birimini** (hangi disk, hangi serial port vb.) belirtir. Bazen ek bayraklar da minor numaraya kodlanır.

```
# Linux SCSI disk minor numarası formülü:
minor = 16 × drivenum + partition
# Örnek: 2. SCSI diskin (drivenum=1) 3. bölümü (partition=3):
minor = 16 × 1 + 3 = 19
```

4.3 mknod ile Cihaz Dosyası Oluşturma

```
# Sözdizimi: mknod [isim] [c|b] [major] [minor]
$ sudo mknod /dev/myserial c 4 64
# dosya adı tür maj min
# Bu komut /dev/myserial adında bir karakter cihaz dosyası oluşturur
# major=4 (Linux'ta tty sürücüsü), minor=64 (birim numarası)
```

5. Cihaza Özgü Davranışlar ve ioctl

Her cihaz birbirinin aynısı değildir. Standart read/write çağrılarının karşılamadığı cihaza özgü işlemler için özel bir mekanizmaya ihtiyaç vardır.

Örnekler:

- Teyp sürücüsü: bir 'dosyayı' atla (skip a file on tape)
- Terminal: backspace, Ctrl+C gibi özel karakterleri işle
- Disk: düşük seviyeli parametreleri ayarla (örn. hdparm)

5.1 ioctl Sistem Çağrısı

Bu ihtiyaçları karşılamak için Unix, **ioctl** (I/O control) sistem çağrısını sunar:

```
#include
int ioctl(int fd, unsigned long request, ...);
// dosya komut kodu isteğe bağlı argüman
// Örnek: terminal pencere boyutunu öğren
struct winsize ws;
ioctl(STDOUT_FILENO, TIOCGWINSZ, &ws);
printf("Satır: %d, Sütun: %d\n", ws.ws_row, ws.ws_col);
```

5.2 TTY Line Discipline

Terminal benzeri tüm cihazlar (tty, pty, serial port) için özel karakter işleme, **line discipline** katmanı tarafından yapılır. Bu katman sürücü ile uygulama arasında oturur ve şunları sağlar:

Line Discipline İşlevi	Açıklama
Satır düzenleme	Backspace, Delete, Ctrl+U ile satırı sil
Sinyal üretme	Ctrl+C → SIGINT, Ctrl+Z → SIGTSTP
Canonical mod	Uygulama satır satır veri alır (Enter bekler)
Raw mod	Her karakter anında iletilir (metin editörleri)
Echo	Yazılan karakteri terminale geri yaz

6. Cihaz Sürücü Arayüzü (Device Driver Interface)

Tüm cihaz sürücüleri, işletim sisteminin kendileriyle konuşabilmesi için standart **entry point** (giriş noktası) fonksiyonlarını uygulamak zorundadır. Bu yapı, sürücüler arası tutarlılığı sağlar ve çekirdeğin sürücüyü tanımasına olanak verir.

Entry Point	Prototip (C)	Açıklama
<code>open</code>	<code>int (*open)(struct inode*, struct file*)</code>	Cihazı başlat, kaynakları ayır
<code>close</code>	<code>int (*release)(struct inode*, struct file*)</code>	Kaynakları serbest bırak
<code>read</code>	<code>ssize_t (*read)(struct file*, char*, size_t, loff_t*)</code>	Cihazdan veri oku
<code>write</code>	<code>ssize_t (*write)(struct file*, const char*, size_t, loff_t*)</code>	Cihaza veri yaz
<code>ioctl</code>	<code>long (*unlocked_ioctl)(struct file*, uint, ulong)</code>	Cihaza özel komutlar
<code>mmap</code>	<code>int (*mmap)(struct file*, struct vm_area_struct*)</code>	Belleğe eşle (isteğe bağlı)
<code>poll</code>	<code>uint (*poll)(struct file*, poll_table*)</code>	Olay bekleme (select/poll)

i Unix Semantiği

Sürücüler, Unix dosya semantiğini mümkün olduğunca uygular. Ancak cihaza özgü kısıtlamalar geçerliliğini korur — örneğin bir disk sürücüsü rastgele seek desteklerken bir klavye sürücüsü desteklemez.

```
/* Linux karakter sürücüsü – file_operations yapısı */
static struct file_operations mydev_fops = {
    .owner = THIS_MODULE,
    .open = mydev_open,
    .release = mydev_close,
    .read = mydev_read,
    .write = mydev_write,
    .unlocked_ioctl = mydev_ioctl,
};
/* register_chrdev ile çekirdeğe kayıt */
register_chrdev(major, "mydev", &mydev_fops);
Linux çekirdek sürücüsü — file_operations yapısı örneği
```

7. Blok Cihazlar (Block Devices)

Blok cihazlar karakter cihazlardan farklı bir arayüze sahiptir. Temel işlev diskten blok okuma/yazmadır.

- Standart bir arayüze sahiptirler, ancak karakter cihaz arayüzünden farklıdır
- Buffer cache (tampon ön bellek) ve bellek alt sistemiyle **entegre çalışır**

- Ağırlıklı olarak dosya sistemi üzerinden erişilir
- İstisna: bazı yüksek performanslı veritabanları (raw I/O) diski doğrudan kullanır
- Hâlâ cihaza özgü komutlara ihtiyaç duyulur — Linux'ta **hdparm** komutu

```
# hdparm ile disk parametrelerini görüntüle
$ sudo hdparm -I /dev/sda
# Buffer cache'i bypass ederek okuma hızını test et (raw I/O)
$ sudo hdparm -t --direct /dev/sda
# Read-ahead boyutunu ayarla (KB cinsinden)
$ sudo hdparm -a 256 /dev/sda
```

□ Buffer Cache Entegrasyonu

Buffer cache, sık kullanılan disk bloklarını RAM'de tutar. Blok cihaz sürücüsü, bir okuma isteği geldiğinde önce cache'e bakar; bulamazsa (cache miss) diski okur ve sonucu cache'e ekler. Bu mekanizma disk I/O'sunu dramatik biçimde azaltır.

8. Disk Zamanlaması (Disk Scheduling) — Giriş

Manyetik disk sürücülerinde I/O işleminin süresi üç bileşenden oluşur:

Bileşen	Türkçe	Tipik Değer	Açıklama
Seek time	Arama süresi	1-15 ms	Kolu (arm) doğru silindire taşıma
Rotational delay	Dönme gecikmesi	0-8 ms	Doğru sektörün okuma kafasının altına gelmesi
Transfer time	Transfer süresi	< 1 ms	Verinin aktarımı (genellikle ihmal edilir)

Disk I/O'sunun yavaşlığının ana nedeni mekanik hareketlerdir: seek time + rotational delay toplamı birkaç ms ile onlarca ms arasında değişir. Bu süre, CPU'nun milyonlarca işlem yapabileceği bir zaman dilimidir!

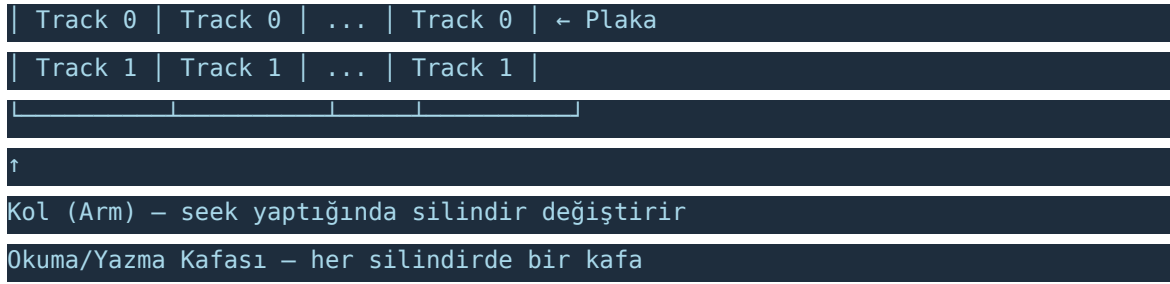
⚠ Neden Optimizasyon Gerekli?

Disk I/O sürücüsü verimi optimize etmek zorundadır. Rastgele sırayla gelen istekleri akıllıca sıralayarak seek süresini minimize etmek, disk performansını birkaç kata kadar artırabilir.

Disk Anatomisi (Basit Diyagram)

Silindir 0 Silindir 1 ... Silindir N





9. Çatışan Gereksinimler (Conflicting Demands)

Disk zamanlaması algoritmaları birbiriyle çatışan üç hedef arasında denge kurmak zorundadır:

Hedef	Tanım	Avantaj	Dezavantaj
Fairness (Adillik)	I/O isteklerini geliş sırasına göre işle	Her istek makul sürede işlenmelidir	Bazı istekler sonsuza kadar ertelenebilir
Efficiency (Verimlilik)	Seek süresini minimize et	Yüksek throughput	Geliş sırası göz ardı edilir
Promptness (Zamanındalık)	Hiçbir isteği indefinitely geciktirme	Starve (açlık) engellenir	Verimliliği düşürebilir

⚠ Trade-off

Hiçbir algoritma üç hedefi aynı anda tam olarak karşılayamaz. İyi bir disk zamanlama algoritması bu üç hedef arasında makul bir denge kurar.

10. FCFS — İlk Gelen İlk Hizmet (First Come, First Served)

En basit disk zamanlaması algoritmasıdır. I/O istekleri geliş sırasına göre işlenir.

Özellik	Değerlendirme
Adillik	☐ Mükemmel — hiçbir istek atlanmaz
Verimlilik	☐ Çok kötü — rastgele seek örüntüsü
Zamanındalık	☐ Garantili sıra
Uygulama kolaylığı	☐ Çok basit — sadece kuyruk

Kuyruktaki istekler: **98, 183, 37, 122, 14, 124, 65, 67** (başlangıç konumu: **53**)

```
Sıra: 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
Hareket (silindir): 45 85 146 85 108 110 59 2
Toplam kol hareketi: 45+85+146+85+108+110+59+2 = 640 silindir
FCFS örneği — 640 silindir toplam hareket
```

❑ FCFS'nin Sorunu

FCFS, kol çok uzak konumlar arasında ileri geri gidip gelebilir. Bu thrashing benzeri bir davranış yaratır ve disk throughput'unu düşürür. Üretim sistemlerinde nadiren tercih edilir.

11. SSF — En Kısa Seek Önce (Shortest Seek First)

Her adımda mevcut kol konumuna en yakın bekleyen isteği seçer. Bu greedy (açgözlü) bir stratejidir.

Özellik	Değerlendirme
Adillik	❑ Kötü — disk kenarlarındaki istekler sürekli ertelenebilir
Verimlilik	❑ İyi — seek süresini minimize eder
Starvation	❑ Risk var — ortadaki istekler hep önce seçilirse kenar istekler açlık çeker

Aynı kuyruk (98, 183, 37, 122, 14, 124, 65, 67, başlangıç: 53):

```
Adım 1: 53 → En yakın: 65 (mesafe: 12)
Adım 2: 65 → En yakın: 67 (mesafe: 2)
Adım 3: 67 → En yakın: 37 (mesafe: 30)
Adım 4: 37 → En yakın: 14 (mesafe: 23)
Adım 5: 14 → En yakın: 98 (mesafe: 84)
Adım 6: 98 → En yakın: 122 (mesafe: 24)
Adım 7: 122 → En yakın: 124 (mesafe: 2)
Adım 8: 124 → En yakın: 183 (mesafe: 59)
Sıra: 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183
Toplam kol hareketi: 12+2+30+23+84+24+2+59 = 236 silindir
SSF örneği — sadece 236 silindir (FCFS'ye göre %63 daha az)
```

❑ SSF'nin Starvation Sorunu

SSF çok verimlidir ancak ortaya adil değildir. Disk kenarlarındaki (silindir 0 veya N) istekler, ortaya yakın sürekli yeni istekler geldiğinde sonsuza kadar bekleyebilir. Bu duruma indefinite postponement (sonsuz erteleme) veya starvation denir.

12. Elevator Algorithm (SCAN / Asansör Algoritması)

Asansör algoritması, hem FCFS'nin **verimsizliğini** hem de SSF'nin **starvation sorununu** aşmak için geliştirilmiştir. Bir asansörün çalışmasını model alır: **tek yönde hareket eder**, aynı yol üzerindeki tüm istekleri yerine getirir, son noktaya ulaşınca yönünü değiştirir.

□ Elevator Algoritması — Temel Kural

Kolu tek yönde hareket ettir. O yöndeki tüm istekleri sırayla işle. Uçta ters yöne geç ve tekrarla.

12.1 Nasıl Çalışır?

- Kol **bir yönde** hareket eder (örn. artan silindir numaraları)
- Bekleyen I/O istekleri **sektör numarasına göre sıralanır**
- Hareket yönünde olan istekler sırayla işlenir
- O yönde başka istek kalmayınca **yön değiştirilir** ve diğer yöndeki istekler işlenir
- Bu sayede tüm istekler **sınırlı bekleme süresiyle** işlenir

12.2 Örnek Çalışma

Kuyruk: **98, 183, 37, 122, 14, 124, 65, 67**, başlangıç: **53**, yön: **artan (yukari)**:

Sıralanmış istekler: 14, 37, 65, 67, 98, 122, 124, 183

Artan yönde (53'ten büyük olanlar önce):

53 → 65 → 67 → 98 → 122 → 124 → 183 (uca ulaş)

Yön değiştir (azalan):

183 → 37 → 14

Hareketler: 12 2 31 24 2 59 | 146 23

Toplam: 12+2+31+24+2+59+146+23 = 299 silindir

Elevator / SCAN örneği — 299 silindir toplam hareket

12.3 Görsel Diyagram

Silindir	Yön	İşlem	Birikimli Hareket
53 (başlangıç)	→ artan	Başla	0
65	→ artan	İste işle	12
67	→ artan	İsteği işle	14
98	→ artan	İsteği işle	45
122	→ artan	İsteği işle	69
124	→ artan	İsteği işle	71
183	→ artan ← DÖNDÜ	İsteği işle	130
37	← azalan	İsteği işle	276
14	← azalan	İsteği işle	299

12.4 Elevator Algoritmasının Özellikleri

Özellik	Değerlendirme	Açıklama
Adillik	□ Makul	Her istek en fazla bir tam tur bekler
Verimlilik	□ İyi	SSF'ye yakın performans, ancak daha adil
Starvation	□ Yok	Kol her bölgeden düzenli geçer
Uygulama	□ Kolay	Sadece sıralama ve yön takibi gerekli
Fairness sorunu	△ Var	Kola yakın bölgeler daha sık servis görür

12.5 Adillik Sorunu

Elevator algoritmasının hâlâ bir adillik sorunu vardır:

△ Elevator'ın Adillik Sorunu

Eğer kolun mevcut konumuna yakın çok sayıda istek gelirse, bu istekler tekrar tekrar önce işlenir. Disk kenarlarındaki (extremes) istekler az öncelik kazanır. Özellikle dosya sistemleri blokları yerel olarak ayırmaya çalıştığından (lokalite ilkesi), kolun merkezine yakın bölgeler daha avantajlıdır.

Bu sorunu çözmek için gelişmiş varyantlar geliştirilmiştir:

Varyant	Açıklama	Avantaj
LOOK	Uca gitmek yerine son isteğe kadar git, yön değiştir	Gereksiz boş seek yok
C-SCAN	Sadece bir yönde servis ver, dönüşte hızlıca başa dön	Daha uniform bekleme
C-LOOK	C-SCAN + LOOK kombinasyonu	En pratik varyant
DEADLINE	Her isteğe son kullanma tarihi (deadline) ata	Real-time I/O garantisi

12.6 Algoritma Karşılaştırması

Aynı örnek kuyruk (**98, 183, 37, 122, 14, 124, 65, 67**, başlangıç: **53**) için tüm algoritmaların karşılaştırması:

Algoritma	İşlem Sırası (başlangıç: 53)	Toplam Hareket	Değerlendirme
FCFS	53→98→183→37→122→14→124→65→67	640 sil.	❌ Çok kötü
SSF	53→65→67→37→14→98→122→124→183	236 sil.	⚠ Starvation riski
Elevator	53→65→67→98→122→124→183→37→14	299 sil.	✅ Dengeli

❓ Neden Elevator?

Elevator algoritması, FCFS'den çok daha verimli (%53 daha az seek) ve SSF'den çok daha adildir. Bu denge, onu pratik sistemler için tercih edilen başlangıç noktası yapar. Linux cfq, deadline ve mq-deadline scheduler'ları bu fikri temel alır.

13. Quiz Soruları ve Cevapları

i Kullanım Rehberi

Aşağıdaki sorular sınav formatındadır. Cevapları okumadan önce kendiniz çözmeye çalışın.

S1. Unix sisteminde /dev/sda3 cihaz dosyasını incelerken major numarasının 8, minor numarasının 3 olduğunu görüyorsunuz. Bu bilgilerden ne anlarsınız? Major ve minor numaralarının işlevlerini açıklayınız.

- Major numarası 8, bu cihazın kernel içinde hangi cihaz sürücüsüyle (device driver) yönetileceğini belirtir. Linux'ta major 8 = SCSI/SATA disk sürücüsüdür.
- Minor numarası 3, birim ve bölümü (unit/partition) belirtir. Linux SCSI formülüne göre: $\text{minor} = 16 \times \text{drivenum} + \text{partition}$ → $3 = 16 \times 0 + 3$, yani ilk diskin (sda) 3. bölümü (sda3).
- Bu dosya bir blok cihaz (block device) olduğundan 'b' harfiyle başlar.
- Çekirdek, bu dosyaya yapılan sistem çağrısını major numarasına göre ilgili sürücüye yönlendirir, minor numarasını ise sürücüye parametre olarak iletir.

S2. Disk zamanlamasında 'starvation' (açlık) nedir? FCFS bu sorunu yaşar mı? SSF'de neden bu sorun var? Açıklayınız.

- Starvation: Bir I/O isteğinin diğer istekler sürekli öncelik kazandığı için belirsiz süre beklemek zorunda kalması durumudur.
- FCFS'de starvation yoktur. Her istek geliş sırasına göre işlenir; hiçbir istek atlanmaz.
- SSF'de starvation riski vardır. SSF her adımda kola en yakın isteği seçer (greedy). Disk kolunun bulunduğu bölgeye yakın istekler sürekli gelirse, diskin kenarlarındaki (uç silindirlerdeki) istekler hiçbir zaman seçilemeyebilir.
- Örnek: Kol 50. silindir civarındayken, 40, 45, 55, 60 nolu istekler sürekli gelirse, 0 veya 200 nolu silindirdeki istekler sonsuza kadar bekler.

S3. Disk zamanlaması için Elevator algoritması önerilmektedir. Aşağıdaki istek kuyruğunu Elevator algoritması ile çözünüz ve toplam kol hareketini hesaplayınız. Başlangıç konumu: 100, Yön: artan Kuyruk: 55, 58, 39, 18, 90, 160, 150, 38, 184

- Önce tüm istekleri sırala: 18, 38, 39, 55, 58, 90, 150, 160, 184
- Artan yönde (100'den büyük olanlar): 150 → 160 → 184 (uca ulaş, yön değiştir)
- Azalan yönde: 90 → 58 → 55 → 39 → 38 → 18
- Hareket hesabı:
- 100→150: 50 | 150→160: 10 | 160→184: 24
- 184→90: 94 | 90→58: 32 | 58→55: 3 | 55→39: 16 | 39→38: 1 | 38→18: 20
- Toplam: 50+10+24+94+32+3+16+1+20 = 250 silindir

S4. FCFS, SSF ve Elevator algoritmalarını adillik, verimlilik ve starvation açısından karşılaştırınız.

- FCFS: En adil (FIFO sırası), en verimsiz (rastgele seek), starvation yok.
- SSF: En verimli (greedy, minimum seek), en az adil (kenar istekler starve edebilir), starvation riski VAR.
- Elevator: Orta düzey verimlilik (FCFS'den iyi, SSF'ye yakın), makul adillik (her bölge düzenli servis görür), starvation YOK — ancak kolun geçtiği bölgeler daha sık servis görür (kısmi adillik sorunu).
- Sonuç: Gerçek sistemler için Elevator (veya türevleri) en iyi dengeyi sunar.

S5. Unix'te bir cihaz dosyası oluşturmak için hangi komut kullanılır? /dev/myblock adlı, major numarası 10, minor numarası 5 olan bir blok cihaz dosyası oluşturan komutu yazınız ve her parametreyi açıklayınız.

- Komut: `sudo mknod /dev/myblock b 10 5`
- /dev/myblock → oluşturulacak dosyanın yolu
- b → blok cihaz (block device); karakter cihaz için 'c' kullanılır
- 10 → major numarası — kullanılacak cihaz sürücüsünü belirler
- 5 → minor numarası — cihaz birimi (unit) veya bölüm (partition) bilgisi

S6. ioctl sistem çağrısının amacını açıklayınız. Standart read/write çağrıları ile karşılanamayan, ioctl gerektiren iki gerçek hayat örneği veriniz.

- ioctl (I/O control): Standart dosya işlem çağrılarıyla (read/write) gerçekleştirilemeyen, cihaza özgü kontrol ve yapılandırma işlemleri için kullanılır.
- Örnek 1: Terminal pencere boyutunu öğrenmek. read/write bunu yapamaz; ioctl(fd, TIOCGWINSZ, &ws;) çağrısıyla satır/sütun sayısı alınır.
- Örnek 2: Teyp sürücüsünde bir 'dosyayı' atlamak (MTFSF — forward space file). Teyp konumu fiziksel bir işlemdir; read ile değil ioctl ile yapılır.
- Diğer örnekler: disk SMART bilgileri almak (hdparm), ağ kartı özelliklerini sorgulamak (SIOCGIFFLAGS), ses kartı örnekleme hızını ayarlamak.

S7. Disk I/O süresinin üç bileşenini listeleyin ve bunları büyüklüklerine göre sıralayın. Hangisi en dominant bileşendir ve neden?

- 1. Seek time (Arama süresi): Kolun doğru silindire taşınması. Tipik: 1-15 ms (ortalama ~5-8 ms). Mekanik hareketten dolayı en yavaş ve en dominant bileşendir.
- 2. Rotational delay (Dönme gecikmesi): Doğru sektörün okuma kafasının altına gelmesi. Tipik: 0-8 ms (ortalama ~4 ms, 7200 RPM disk için).
- 3. Transfer time (Veri transfer süresi): Verinin platadan kafaya okunması. Tipik: < 1 ms. Genellikle ihmal edilir.
- En dominant: Seek time. Çünkü kolu hareket ettirmek fiziksel (mekanik) bir işlemdir; bu süre CPU veya bellek işlemlerine kıyasla binlerce kat yavaştır. Disk zamanlaması algoritmalarının temel amacı seek time'ı minimize etmektir.

S8. 'Elevator algoritmasının hâlâ adillik sorunu vardır' ifadesini açıklayınız. Bu sorun hangi durumlarda ortaya çıkar?

- Elevator algoritması starvation'ı önler (her istek en fazla bir tam tur bekler), ancak eşit muamele (fairness) garantisi vermez.
- Sorun: Kolun mevcut konumuna yakın bölgeler daha sık servis görür. Kola yakın sürekli yeni istekler geldiğinde, aynı yakın bölge defalarca servis alırken uzak bölgedeki istekler daha uzun bekler.
- Özellikle: Dosya sistemleri blokları yerel olarak (disksel bölgede) ayırmaya çalışır (locality principle). Bu nedenle belirli silindir aralıklarından yoğun istek gelir ve o bölge sürekli 'taze' isteklerle dolu kalır.
- Çözüm yaklaşımları: C-SCAN (dairesel tarama — tek yönlü, başa dönerken servis verme), DEADLINE scheduler (her isteğe maksimum bekleme süresi sınırı), CFQ (Completely Fair Queuing — süreç bazlı adil kuyruk).

14. Kitap Bölüm Sonu Soruları ve Çözümleri

Aşağıdaki sorular Tanenbaum 'Modern Operating Systems' kitabının I/O ve Disk Zamanlaması bölümünden seçilmiştir.

❏ Soru 1

Disk zamanlaması için en uygun algoritma hangisidir? FCFS, SSF ve Elevator'ı değerlendirerek argümanınızı destekleyin.

Çözüm:

Tek bir 'en uygun' algoritma yoktur; bağlama (context) göre tercih değişir.

- FCFS: Düşük yükte adil ve yeterli. Yoğun I/O yükünde verimsiz.
- SSF: Throughput-odaklı sistemlerde (batch işleme) iyi. Gerçek zamanlı veya çok kullanıcıli sistemlerde starvation riski nedeniyle uygun değil.
- Elevator (SCAN): Genel amaçlı sistemler için en dengeli seçimdir. Hem makul verimlilik hem de starvation yokluğu sağlar.

Sonuç: Çoğu üretim sistemi Elevator varyantlarını (C-SCAN, DEADLINE, CFQ) tercih eder. Linux mq-deadline ve bfq scheduler'ları bu kategoridedir.

Soru 2

Aşağıdaki disk istek kuyruğu verilmiştir: 176, 79, 34, 60, 92, 11, 41, 114
Başlangıç konumu: 50, Yön: artan SSF ve Elevator algoritmalarını uygulayın. Her ikisi için toplam kol hareketini hesaplayın.

Çözüm:

SSF Çözümü:

50→60 (10) → 60→41 (19) → 41→34 (7) → 34→11 (23) → 11→79 (68) → 79→92 (13) → 92→114 (22) → 114→176 (62)

SSF Toplam: $10+19+7+23+68+13+22+62 = 224$ silindir

Elevator Çözümü (artan yönde başla):

Sıralı liste: 11, 34, 41, 60, 79, 92, 114, 176

Artan yönde (50'den büyük): 60 → 79 → 92 → 114 → 176 (uca ulaş, yön değiştir)

Azalan yönde: 41 → 34 → 11

50→60 (10) → 60→79 (19) → 79→92 (13) → 92→114 (22) → 114→176 (62) → 176→41 (135) → 41→34 (7) → 34→11 (23)

Elevator Toplam: $10+19+13+22+62+135+7+23 = 291$ silindir

Not: Bu örnekte SSF daha iyi sonuç vermiştir. Ancak bu, SSF'nin her zaman daha iyi olduğunu göstermez — starvation riski hâlâ mevcuttur.

Soru 3

Bir Unix sisteminde /dev/sdb ve /dev/sdb1 arasındaki fark nedir? Bu iki dosya için major ve minor numaralarının nasıl farklılaşacağını açıklayınız.

Çözüm:

/dev/sdb: İkinci SCSI/SATA diskinin tamamını (raw disk) temsil eder. Bölümleme tablosu (partition table) dahil tüm diske erişim sağlar.

/dev/sdb1: İkinci diskin birinci bölümünü (partition 1) temsil eder.

Major numarası: Her ikisi de 8 (SCSI disk sürücüsü) — aynı sürücü her ikisini yönetir.

Minor numarası farkı:

/dev/sdb → minor = $16 \times 1 + 0 = 16$ (drivenum=1, partition=0=tüm disk)

/dev/sdb1 → minor = $16 \times 1 + 1 = 17$ (drivenum=1, partition=1)

Benzer şekilde: sdb2 → 18, sdb3 → 19, vb.

Soru 4

Disk zamanlamasında seek time neden rotational delay'den daha dominant bir faktördür? Gerçek disk değerleriyle destekleyin.

Çözüm:

Seek time dominant nedenleri:

1. Seek time tamamen mekanik harekete dayalıdır: kolun fiziksel olarak silindir üzerinde hareket etmesi gerekir.
2. Seek time aralığı çok geniştir: 1 ms (bitişik silindir) ile 15+ ms (disk ucundan uca) arası.

Gerçek değerler (7200 RPM, SATA HDD için):

Ortalama seek time: ~5-8 ms

Ortalama rotational delay: $60/7200 \times 1000 / 2 \approx 4.17$ ms (yarım tur)

Transfer time (64 KB, 100 MB/s): ~0.64 ms

Toplam: ~10-13 ms; bunun ~50-60%'ı seek time.

Bu yüzden disk zamanlaması öncelikle seek time'ı minimize etmeye odaklanır.

Soru 5

C-SCAN algoritması nedir? Standart Elevator (SCAN)'dan farkı nedir? Hangi senaryolarda C-SCAN tercih edilir?

Çözüm:

C-SCAN (Circular SCAN): Elevator algoritmasının dairesel (tek yönlü) varyantıdır.

Fark:

SCAN: Kol bir yönde gider, uca ulaşınca yön değiştirip geri döner ve dönüş yolundaki istekleri de işler.

C-SCAN: Kol sadece bir yönde (artan) istekleri işler. Uca ulaşınca hızlıca (servis vermeden) başa döner ve tekrar artan yönde tarar.

C-SCAN'ın avantajı:

Bekleme süresi daha uniform (eşdağılımlı) dır. SCAN'da, kolun dönüş yolundaki bölgeler yakın zamanda servis görmüş olduğundan düşük bekleyenlerin aksine, henüz bekleyen istekler geri yolda öncelik kazanır — bu düzensiz bekleme yaratır.

C-SCAN'da her istek maksimum bir tam tur bekler.

Tercih senaryosu: Yoğun, yeknesak I/O yükü olan sistemler (sunucu, veritabanı) — bekleme süresinin öngörülebilir olması istenen durumlar.

Özet — Anahtar Kavramlar

Kavram	Özet
I/O Modelleri	Unix: block + character cihazlar. Windows: bitmap modeli. Network: ayrı model. Hedef: device independence.
Major / Minor Numaralar	Major → sürücü seçimi. Minor → birim/bölüm. Farklı namespace'ler. mknod ile oluşturulur.
ioctl	Standart read/write ile yapılamayan cihaz kontrol işlemleri. TTY line discipline ek katman sağlar.
Device Driver Interface	open, close, read, write, ioctl entry point'leri. Unix semantiği uygulanır.
Disk I/O Bileşenleri	Seek time (dominant) + Rotational delay + Transfer time.
FCFS	Adil, verimsiz, basit. Starvation yok. Düşük yük için yeterli.
SSF	Verimli (greedy), adaletsiz, starvation riski var. Yüksek yükte sorunlu.
Elevator (SCAN)	Dengeli: verimli + adil + starvation yok. Kola yakın bölgeler avantajlı (kısmi fairness sorunu).
C-SCAN / C-LOOK	Elevator varyantları. Tek yönlü tarama → daha uniform bekleme süresi.